

Khulna University of Engineering & Technology (KUET)

Department of Computer Science and Engineering

Project Proposal

**NightWatch: A Closed-Loop Graphics Rendering and Spatial Image
Processing Pipeline**

Course No: CSE 4102

Course Title: Computer Graphics and Image Processing Laboratory

Submitted by

Khadimul Islam Mahi

Roll: 2107076

Year: 4th, Semester: 1st, Section: B

Department of Computer Science and Engineering, KUET

September 2026

Abstract

This proposal outlines the design and implementation of NightWatch, an integrated graphics rendering and spatial image processing pipeline. Unlike standard single-domain projects, NightWatch unifies 3D computer graphics with digital image processing by treating a mathematically generated rendered frame as raw sensory data. The system constructs a physically grounded 3D nighttime compound utilizing raw OpenGL, complete with hierarchical kinematics, Bézier curve-based pathfinding, and multi-source light attenuation. A Framebuffer Object (FBO) functions as the inter-pipeline bridge, capturing both color and depth textures. These matrices are subsequently processed in real time by a custom digital image processing module that simulates sensor degradation, applies spatial-domain convolution filtering, and leverages the depth map to compute dynamic depth-of-field effects.

1. Introduction

Computer Graphics (CG) and Digital Image Processing (DIP) are fundamentally complementary fields; CG synthesizes image data from mathematical models, while DIP extracts information from or enhances existing image matrices. NightWatch integrates these domains into a single interactive pipeline. The system models a secure nighttime compound, rendering it from the perspective of an active CCTV camera. Instead of rendering directly to the viewport, the synthetic output is intercepted and mathematically degraded to simulate real-world sensor noise. A custom DIP pipeline is then employed to computationally reconstruct and enhance the visual feed.

2. System Architecture

The system architecture operates in a continuous loop, divided into three distinct stages:

1. Graphics Rendering Engine: Synthesizes the pristine 3D scene from geometric primitives and parametric equations.
2. The FBO Bridge: Flattens the 3D space into 2D data arrays (Color and Depth textures).
3. Image Processing Pipeline: Degrades, filters, and reconstructs the spatial data in real time.

3. Graphics Rendering Pipeline

The first half of the pipeline focuses on building a rigorous 3D environment without relying on external assets, emphasizing foundational mathematics.

3.1 Procedural Geometry and Hierarchical Kinematics

All scene elements (boundary walls, guard hut, truck) are procedurally generated using fundamental geometric primitives (cubes, cylinders, spheres). Complex entities employ hierarchical transformations. The CCTV camera utilizes a 4-degree-of-freedom kinematic chain (Base $\rightarrow$ Yaw Joint $\rightarrow$ Pitch Joint $\rightarrow$ Lens), allowing transformation matrices to propagate accurately.

3.2 Bézier Curve Motion Planning

Dynamic entities traverse the scene along parametrically defined Bézier curves. The position $P(t)$ is evaluated iteratively, and the derivative (tangent vector) $P'(t)$ is computed to dynamically orient the models along the curve, ensuring realistic non-linear traversal.

3.3 Illumination and Shading Models

The scene employs a sophisticated tri-source lighting system:

- Moonlight: Directional lighting with infinite distance.
- Sodium Security Lamp: A point light utilizing realistic quadratic attenuation (see Equation 1), where d is the distance from the source, and a , b , c are the constant, linear, and quadratic attenuation coefficients.
- IR Illuminator: A dynamic spotlight constrained by a cutoff cone, coupled with the CCTV kinematic chain.

To highlight surface normals and specular reflections, the truck chassis supports real-time computational toggling between Flat, Gouraud, and Phong shading models.

4. Inter-Pipeline Bridge: Framebuffer Objects

To transition from synthesis to analysis, the OpenGL default framebuffer is bypassed. The scene is rendered into a custom Framebuffer Object (FBO) with two bound textures: a standard RGBA Color Texture and a grayscale Depth Texture. The depth map effectively encodes the Z-buffer, storing the normalized spatial distance of every fragment from the view plane.

5. Spatial Image Processing Pipeline

The 2D matrices generated by the FBO serve as the raw input for the DIP engine.

5.1 Degradation Model

To simulate a low-fidelity security feed, the pristine frame undergoes intentional degradation. Gaussian noise is superimposed mathematically via fragment shaders, and pixel intensities are strictly dampened to mimic the limitations of low-light optical sensors.

5.2 Spatial Convolution and Enhancement

A series of spatial domain filters are applied to the degraded feed. Utilizing custom 3×3 and 5×5 convolution matrices, the pipeline applies Gaussian smoothing to suppress the injected noise. Histogram equalization algorithms dynamically stretch the contrast of the dark feed to reconstruct visibility.

5.3 Depth-Driven Depth of Field

Leveraging the Depth Texture acquired from the FBO, a spatially varying convolution filter is applied. The blur radius R for a pixel at spatial coordinates (x, y) is calculated as a function of its

depth $Z(x, y)$, where Z_{focus} is the dynamic focal plane and α is a constant scaling factor, as shown in Equation 2. This mathematical approach transforms a standard 2D blur convolution into a 3D-aware depth-of-field simulation.

$$\text{Equation 2: } R(x, y) = \alpha \cdot |Z(x, y) - Z_{focus}|$$

6. Expected Outcomes

By the conclusion of this project, NightWatch will successfully demonstrate a real-time, bidirectional workflow between CG synthesis and DIP analysis. The final deliverable will evaluate the computational overhead of executing dual rendering and processing pipelines at interactive framerates, while providing a visual and quantitative analysis of spatial noise reduction techniques.